

Day 1 SQL test scorecard — detailed ratings and feedback per question

DAY 1 RESULTS

6.0
/ 10

Solid foundation — clear gaps to fix

You understand SELECT, filtering, aggregation and window function concepts. Your main gaps are GROUP BY correctness, CTE structure, and query completeness under hard problems.

Correct / passing

4

Partial credit

3

Needs rework

3

QUESTION-BY-QUESTION BREAKDOWN

Q1 — SELECT & filtering

★★★★★ 10/10

Perfect. Correct columns, correct WHERE conditions, correct ORDER BY. Date string comparison works in all major SQL dialects. Clean and concise.

Q2 — GROUP BY & aggregation

★★★☆☆ 4/10

Critical bug: You added `product_id` to GROUP BY. This groups by each individual product, not by category — breaking the intent entirely. Also missing ORDER BY.

```
-- Correct:
SELECT category,
       SUM(amount) AS total_sales_amount,
       COUNT(order_id) AS number_of_orders
FROM orders
GROUP BY category
HAVING COUNT(order_id) > 5
ORDER BY total_sales_amount DESC;
```

Q3 — JOINS

★★★★☆ 5/10

Good instinct with LEFT JOIN. Two bugs: (1) `COUNT(orders.*)` is invalid syntax — use `COUNT(orders.order_id)`. (2) Missing GROUP BY — without it you get one row, not one per customer.

```
SELECT c.name,
       COUNT(o.order_id) AS total_order_count
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;
```

Q4 — window functions

★★★★☆ 9/10

Excellent. Correct use of DENSE_RANK with PARTITION BY and ORDER BY. Minor: using `_rank` as alias is fine but `salary_rank` is cleaner. Also missing the semicolon at the end — small but matters in pipelines.

Q5 — subqueries

★★★★☆ 8/10

Solid approach — derived table subquery is valid and correct. One note: `products.name` in SELECT is fine, but since you aliased the joined table as `products`, be explicit. A correlated subquery is also elegant here and worth knowing:

```
-- Correlated subquery alternative:
SELECT name, category, price
FROM products p
WHERE price > (
  SELECT AVG(price)
  FROM products
  WHERE category = p.category
);
```

Q6 — CTEs

★★★☆☆ 3/10

Multiple issues: (1) You wrote two separate SELECT statements — SQL doesn't allow that without UNION or separate queries. (2) Using a window function SUM OVER to aggregate is wrong here — you need GROUP BY SUM. (3) Typo: `YEAR FRM`. (4) LIMIT inside a CTE is non-standard. (5) The CTE is not referenced correctly for 2024 logic.

```
WITH spend_2023 AS (
  SELECT customer_id, SUM(amount) AS total_spend
  FROM orders
  WHERE EXTRACT(YEAR FROM order_date) = 2023
  GROUP BY customer_id
  ORDER BY total_spend DESC
  LIMIT 3
),
orders_2024 AS (
  SELECT customer_id, COUNT(order_id) AS orders_in_2024
  FROM orders
  WHERE EXTRACT(YEAR FROM order_date) = 2024
  GROUP BY customer_id
)
SELECT s.customer_id, o.orders_in_2024
FROM spend_2023 s
LEFT JOIN orders_2024 o ON s.customer_id = o.customer_id;
```

Q7 — NULL handling

★★★★☆ 6/10

Logic mismatch: You used COALESCE on `manager_id`, but the question asks to replace NULL `bonus` with 0. Also, since WHERE filters for `manager_id IS NULL`, COALESCE on `manager_id` always returns 0 — pointless. Apply COALESCE to `bonus` instead.

```
SELECT id, name, salary, COALESCE(bonus, 0) AS bonus
FROM employees
WHERE manager_id IS NULL;
```

Q8 — date functions

★★★★☆ 5/10

Good thinking — you knew to use LAG and two CTEs. But the first CTE uses SUM as a window function over rows rather than GROUP BY month, which gives a running total per row, not one total per month. The LAG in CTE 2 also has no proper date column to order by.

```
WITH monthly AS (  
  SELECT  
    DATE_TRUNC('month', sale_date) AS month,  
    SUM(amount) AS revenue  
  FROM sales  
  WHERE EXTRACT(YEAR FROM sale_date) = 2023  
  GROUP BY 1  
)  
SELECT  
  month,  
  revenue,  
  LAG(revenue) OVER (ORDER BY month) AS prev_revenue,  
  ROUND(  
    (revenue - LAG(revenue) OVER (ORDER BY month))  
    / LAG(revenue) OVER (ORDER BY month) * 100, 2  
  ) AS pct_growth  
FROM monthly  
ORDER BY month;
```

Q9 — deduplication

★★★★☆ 3/10

Wrong approach: DELETE * is invalid syntax. Also, your subquery only selects duplicate keys but doesn't identify which specific rows to delete — this would wipe all matching rows, not keep one. The correct strategy uses ROW_NUMBER to tag duplicates, then deletes where rank > 1.

```
-- Standard approach (works in PostgreSQL, SQL Server):  
WITH ranked AS (  
  SELECT order_id,  
    ROW_NUMBER() OVER (  
      PARTITION BY customer_id, product_id, order_date  
      ORDER BY order_id  
    ) AS rn  
  FROM orders  
)  
DELETE FROM orders  
WHERE order_id IN (  
  SELECT order_id FROM ranked WHERE rn > 1  
);
```

Q10 — consecutive days streak

★★★★☆ 5/10

You got the right idea — the classic "date minus row_number = constant group" pattern is exactly correct. But: (1) Missing closing parenthesis in the subquery. (2) Typo: login_date. (3) GROUP BY needs grps too. (4) The SELECT for start date should be MIN(login_date).

```

SELECT user_id, MIN(login_date) AS streak_start
FROM (
  SELECT user_id, login_date,
    (login_date - CAST(ROW_NUMBER() OVER (
      PARTITION BY user_id ORDER BY login_date
    ) AS INT)) AS grp
  FROM user_logins
) t
GROUP BY user_id, grp
HAVING COUNT(*) >= 3
ORDER BY user_id, streak_start;

```

Strengths

You instinctively reach for the right tools — DENSE_RANK for ranking, LEFT JOIN for inclusive queries, CTEs for multi-step logic, and the date-minus-row_number streak trick. Your query structure and formatting are clean. Q1, Q4, and Q5 show you can write production-grade SQL.

Key areas to improve before Day 2

1. GROUP BY discipline — only group by what you're aggregating across, not extra columns.
2. CTE structure — one CTE block, multiple named CTEs separated by commas, one final SELECT.
3. DELETE patterns — always use ROW_NUMBER to isolate duplicate row IDs before deleting.
4. Read questions precisely — Q7's COALESCE target was bonus, not manager_id.
5. Typos in queries — "logn_date", "FRM" — slow down and re-read before submitting.

[Start Day 2 questions ↗](#)